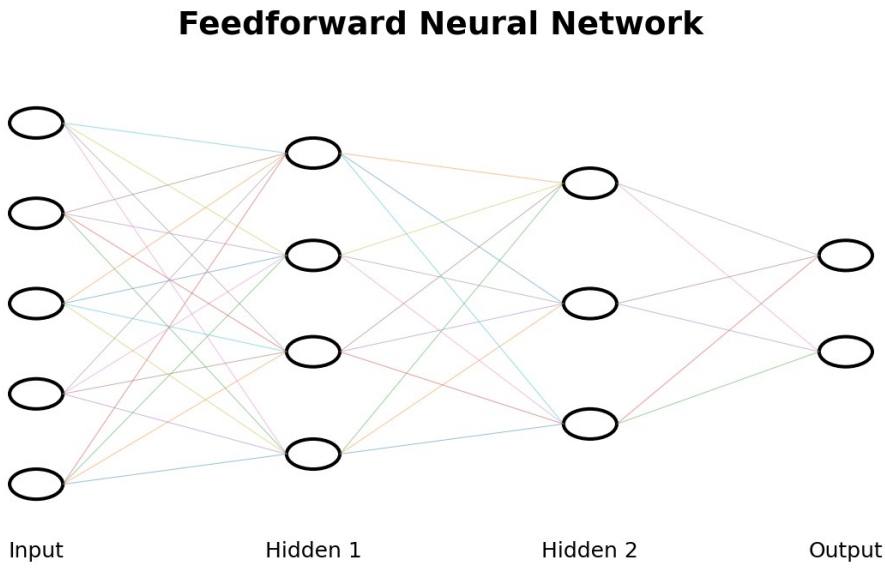
29. DEEP LEARNING

Neural Networks - Backpropagation - Optimization - Regularization



Bu modulun amaci
Deep learningi kutuphane ezberi olarak degil; temsil ogrenme, ileri gecis, kayip fonksiyonu, geri yayilim, optimizasyon ve genelleme mekanizmalari uzerinden anlayarak PyTorch modulune hazir hale gelmek.

Kazanim	Modul sonunda
Temel sezgi	Neuron, layer, activation ve representation learning mantigini aciklayabilirsin.
Egitim mekanizmasi	Forward pass, loss, backpropagation ve optimizer dongusunu kurabilirsin.
Genelleme	Overfitting, dropout, weight decay, early stopping ve data augmentation mantigini kullanabilirsin.
Stabil egitim	Initialization, normalization ve gradient sorunlarini taniyabilirsin.
Mimari farklar	MLP, CNN ve sequence modellerinin hangi veri tiplerine uygun oldugunu ayirt edebilirsin.
Proje dusuncesi	Deep learning problemini veri, metric, architecture ve production acisindan tasarlayabilirsin.

1. Deep Learning nedir?

Deep learning, cok katmanli sinir aglariyla veriden hiyerarsik temsiller ogrenme yaklasimidir. Klasik makine ogrenmesinde feature engineering daha belirgin olabilirken, deep learning ham girdiden yararli representationlari katmanlar boyunca ogrenebilir.

Ana fikir
Model sadece bir fonksiyon yaklasimi degildir; ayni zamanda verinin ise yarayan temsilini de ogrenmeye calisir.

2. Klasik ML ile Deep Learning farki

Boyut	Klasik ML	Deep Learning
Feature	Elle/domain driven daha fazla	Representation model icinde ogrenilebilir
Veri ihtiyaci	Dusuk-orta veride guclu olabilir	Genellikle daha fazla veri ister
Compute	Daha hafif	GPU/accelerator faydali
Tabular	Boosting cok guclu default	Her zaman ilk secim degil
Image/audio/text	Ek feature ihtiyaci yuksek olabilir	DL cok guclu
Yorumlanabilirlik	Genellikle daha kolay	Daha zor

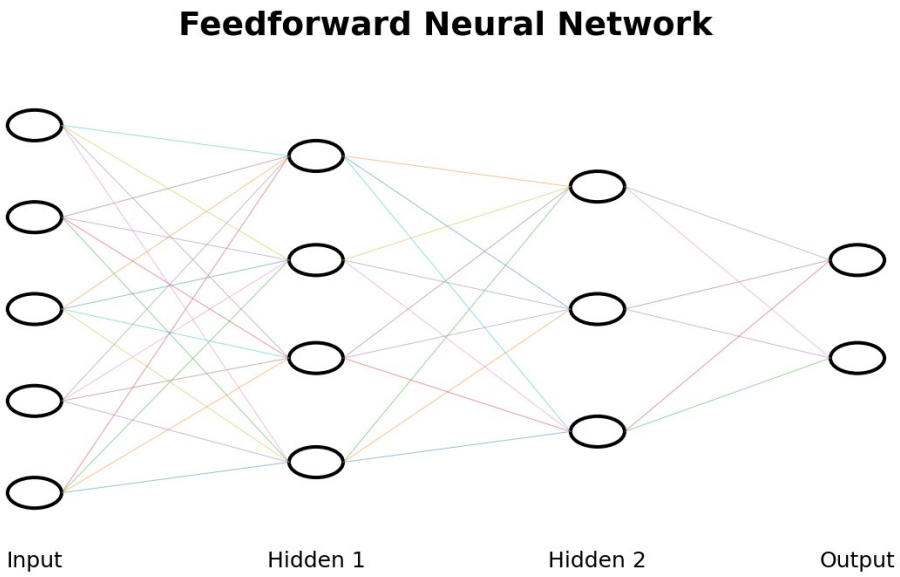
3. Neuron ve lineer donusum

Bir neuron once agirlikli toplam hesaplar, sonra activation uygular: z = w^T x + b, a = f(z).

```
z = X @ W + b
a = activation(z)
```

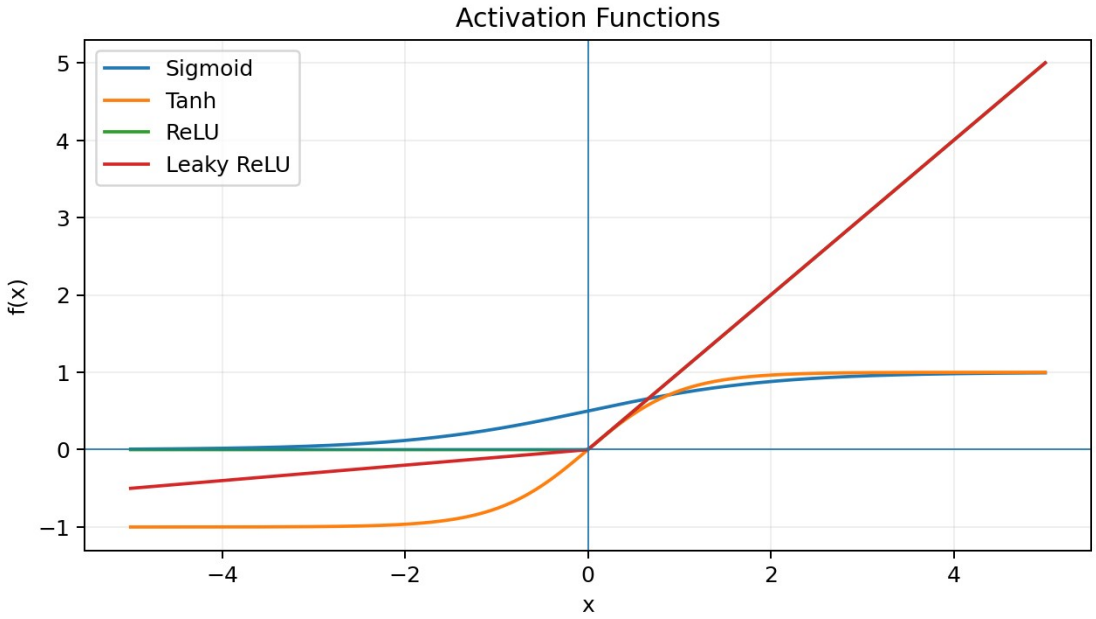
Bias neden var?
Modelin karar yuzeyini orijinden bagimsiz kaydirabilmesini saglar.

4. Layer ve network



- Input layer: ham feature veya onceki representation.
- Hidden layers: ara representationlar.
- Output layer: probleme gore logit, probability veya regression degeri.

5. Activation functions



Activation	Ozellik	Tipik kullanim
ReLU	max(0,x), basit ve hizli	Hidden layers
Leaky ReLU	Negatif bolgede kucuk egim	Dead ReLU riskini azaltma
GELU	Smooth gating benzeri	Transformers
Sigmoid	0-1 arasi	Binary output
Softmax	Sinif olasiliklari toplami 1	Multiclass output

Tanh	-1 ile 1	Bazi recurrent yapilar

Output activation
Activation secimi loss ve target yapisiyla birlikte dusunulmelidir.

6. Forward propagation

Forward pass, girdinin katman katman islenerek predictiona donusmesidir.

```
h1 = relu(X @ W1 + b1)
h2 = relu(h1 @ W2 + b2)
logits = h2 @ W3 + b3
```

7. Loss function

Problem	Yaygin loss	Not
Regression	MSE / MAE / Huber	Outlier duyarliligi farkli
Binary classification	Binary cross-entropy	Logit tabanli uygulama daha stabil olabilir
Multiclass	Cross-entropy	Softmax mantigiyla birlikte
Imbalanced	Weighted CE / focal loss	Business metric yine ayrica izlenmeli

8. Logits ve probability

Logit, output katmaninin activation oncesi ham skorudur. Numerik stabilite nedeniyle modern kutuphaneler loss icinde sigmoid/softmax islemlerini birlestirebilir.

Pratik
Binary classificationda BCEWithLogitsLoss benzeri yapilar genellikle sigmoid + BCEyi ayri yazmaktan daha guvenlidir.

9. Gradient descent

Amac lossu azaltan parametreleri bulmaktir. Basit guncelleme: theta <- theta - learning_rate * gradient.

```
for batch in loader:
    pred = model(batch.x)
    loss = criterion(pred, batch.y)
    loss.backward()
    optimizer.step()
```

10. Backpropagation sezgisi

Backpropagation, chain rule kullanarak output hatasinin her parametreye etkisini ters yonde hesaplar. Her agirlik icin lossun o agirliktaki kucuk degisime ne kadar duyarlı oldugu bulunur.

Backprop = gradient hesabi
Optimizer ise bu gradientleri kullanarak parametreleri gunceller. Iki kavram ayni sey degildir.

11. Chain rule neden kritik?

Cok katmanli model bile birbiri ardina uygulanmis fonksiyonlar zinciridir. Turevler chain rule ile geriye aktarilir.

```
y = f(g(h(x)))
dy/dx = df/dg * dg/dh * dh/dx
```

12. Batch, epoch, iteration

Kavram	Tanım
Batch	Tek optimizer adiminda islenen ornek grubu
Iteration / step	Bir batch icin bir guncelleme
Epoch	Tum training datasinin yaklasik bir kez gorulmesi

13. Batch size trade-off

- Kucuk batch: noisy gradient, az memory, bazen daha iyi generalization.
- Buyuk batch: daha stabil gradient, GPU verimliligi, daha cok memory.
- Learning rate ve batch size birlikte etkilesir.

14. Optimizerlar

Optimizer	Fikir	Not
SGD	Direkt gradient adimi	Basit ve guclu baseline
Momentum	Gecmis yon bilgisini tasir	Osilasyonu azaltabilir
RMSProp	Adaptive step size	RNN tarihinde yaygin
Adam	Momentum + adaptive scaling	Guclu genel default
AdamW	Weight decay ayrik uygulanir	Modern DLde cok yaygin

15. Learning rate

Learning rate egitimin en kritik hyperparameterlarindan biridir. Cok yuksekse loss diverge olabilir; cok dusukse egitim gereksiz yavaslar veya zayif noktada kalabilir.

Ilk bakilacak parametre
Model train olmuyorsa once learning rate, data pipeline, target ve loss uyumunu kontrol et.

16. Learning-rate scheduler

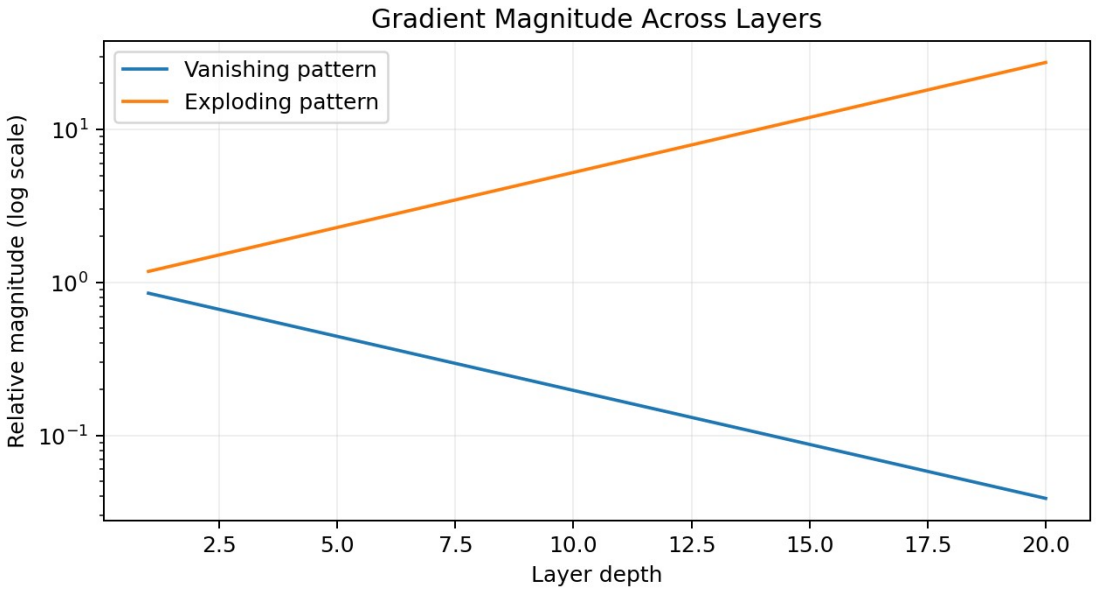
- Step decay
- Cosine annealing
- One-cycle policy
- Warmup + decay

Transformer baglantisi
Warmup, buyuk modellerde egitimin ilk adimlarini stabilize etmek icin yaygin bir tekniktir.

17. Weight initialization

Yontem	Fikir
Xavier/Glorot	Tanh/sigmoid ailesi icin variance koruma
He/Kaiming	ReLU ailesi icin uygun variance
Random tiny	Her durumda ideal degil
Zero weights	Symmetry nedeniyle kotu

18. Vanishing ve exploding gradients



Cozum ailesi
Iyi initialization, ReLU/GELU, normalization, residual connections, gradient clipping ve dikkatli learning rate.

19. Normalization

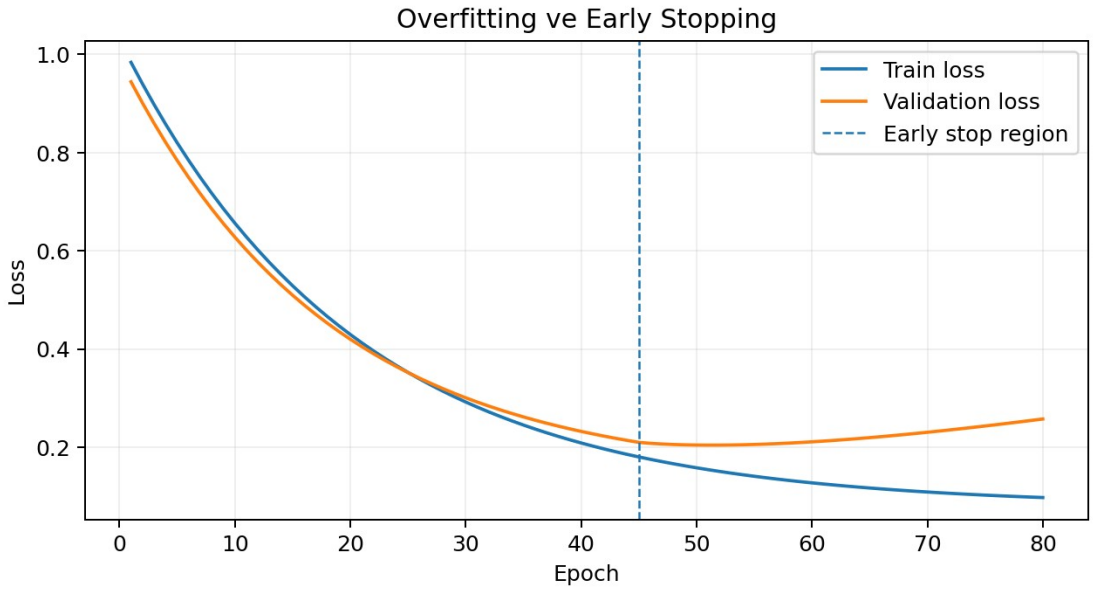
Yontem	Nerede?	Fikir
BatchNorm	CNN/MLP	Batch statistics ile activation scale
LayerNorm	Transformer/RNN	Feature boyutunda normalize
Input scaling	Her yerde	Girdiyi uygun olcege getirir

20. Residual connections

Residual yapida blok f(x) yerine x + f(x) ogrenilir. Derin aglarda gradient akisina yardim eder ve cok derin mimarileri egitilebilir hale getirir.

```
y = x + block(x)
```

21. Overfitting



Training loss azalmaya devam ederken validation loss yukselmeye basliyorsa model training verisini fazla ogreniyor olabilir.

22. Regularization

Teknik	Mekanizma
Weight decay	Buyuk agirliklari cezalandirir
Dropout	Trainingde neuronlarin bir kismini rastgele kapatir
Early stopping	Validation bozuldugunda durur
Data augmentation	Yeni varyasyonlar olusturur
Label smoothing	Asiri kesin targeti yumusatir

23. Dropout

```
# training sirasinda
h = dropout(h, p=0.3)
```

Eval modu
Dropout training ve inference modunda farkli davranir. Frameworkte model.train() ve model.eval() ayrimi bu yuzden kritiktir.

24. Early stopping

Validation metric belirli sayida epoch iyilesmezse egitim durdurulur ve en iyi checkpoint saklanir.

25. Data augmentation

Veri	Ornek augmentation
Image	crop, flip, color jitter
Audio	noise, time shift, masking
Text	dikkatli paraphrase/back translation
Tabular	Genellikle daha problem-spesifik

Kural
Augmentation label semantigini korumali. Gercekci olmayan donusumler noise uretir.

26. Train / validation / test

- Train: parameter optimization.
- Validation: architecture, learning rate, epoch, threshold secimi.
- Test: final unbiased estimate.

Test seti tuning araci degildir
Her yeni denemeden sonra teste bakmak test setini fiilen validationa cevirir.

27. Deep learning metricleri

Problem	Loss	Metric
Binary cls	BCE	PR-AUC, ROC-AUC, F1, recall
Multiclass	Cross-entropy	accuracy, macro F1
Regression	MSE/Huber	MAE, RMSE, R2
Ranking	pairwise/listwise loss	NDCG, Recall@K
Segmentation	Dice/BCE	IoU, Dice

28. Loss ile metric farki

Loss differentiable olup optimizer tarafindan kullanilir. Metric ise business veya model performansini raporlar; differentiable olmak zorunda degildir.

29. Classification threshold

Network probability uretse bile final sinif threshold ile belirlenir. Imbalanced datada 0.5 defaultu genellikle business-optimal degildir.

30. Multilayer Perceptron (MLP)

MLP, dense fully-connected katmanlardan olusan temel neural network mimarisidir. Tabular ve genel vector girdiler icin kullanilabilir.

```
input -> Linear -> ReLU -> Dropout -> Linear -> Output
```

31. Convolutional Neural Networks (CNN)

CNN yerel patternleri paylasilan filtrelerle yakalar. Image verisinde edge, texture ve daha yuksek seviyeli objeler katmanlar boyunca ogrenilebilir.

Kavram	Anlam
Kernel/filter	Yerel pattern dedektoru
Stride	Filtrenin adim boyu
Padding	Uzamsal boyutu kontrol etme
Pooling	Boyut azaltma / invariance
Feature map	Filtre aktivasyonu

32. Sequence modelleri

RNN/LSTM/GRU, sirali veriyi hidden state ile isler. Modern NLPde Transformers buyuk olcude baskin hale gelse de recurrent mantigi zaman serisi ve sequence representation acisindan ogreticidir.

33. RNN ve LSTM sezgisi

- RNN: onceki hidden state yeni girdiye aktarilir.
- LSTM: gate yapilariyla uzun sureli bilgiyi korumaya calisir.
- Gradient sorunlari recurrent aglarda ozellikle onemlidir.

34. Attentiona gecis

Attention, sequence icindeki her tokenin diger tokenlarla iliskisini agirliklandirarak baglam representationi kurar. Bu fikir Transformer ve LLM modulunun temelidir.

35. Embeddings

Categorical/token girdiler one-hot yerine dense vektorlere map edilebilir. Embedding uzayinda benzer entityler yakin representationlar ogrenebilir.

36. Transfer learning

Onceden buyuk dataset uzerinde egitilmis modeli yeni probleme adapte etmektir.

Strateji	Ne zaman?
Feature extractor	Az veri, hizli baseline
Partial fine-tune	Orta veri, domain farki
Full fine-tune	Yeterli veri/compute
Parameter-efficient tuning	Buyuk foundation model

37. Fine-tuning

Baslangic learning rate
Pretrained modeli fine-tune ederken genellikle sifirdan egitime gore daha kucuk learning rate kullanilir.

38. Checkpointing

- Model weights
- Optimizer state
- Scheduler state
- Epoch/step
- Random seed / config

Reproducibility
Sadece model.pt saklamak deneyin tamamini yeniden uretebilmek icin yeterli olmayabilir.

39. Reproducibility

```
seed = 42
random.seed(seed)
np.random.seed(seed)
# framework seedlerini de ayarla
```

Tam determinism maliyetli olabilir
GPU kernel secimleri ve distributed training tam determinismi zorlastirabilir. Ama config, seed ve veri versiyonu mutlaka kaydedilmelidir.

40. GPU mantigi

Deep learning yogun matrix/tensor islemleri yapar. GPU cok sayida paralel aritmetik islemi nedeniyle egitimi hizlandirabilir.

Bottleneck	Cozum
GPU memory	Batch size / mixed precision
Data loading	Parallel workers / prefetch
CPU->GPU transfer	Pinned memory / batch transfer
Compute	Mixed precision / efficient kernels

41. Mixed precision

FP16/BF16 gibi dusuk hassasiyetli hesaplar memory ve compute avantajı saglayabilir. Loss scaling gibi tekniklerle numerik stabilite korunur.

42. Gradient clipping

Gradient normu belirli esigi asinca olceklenir. Ozellikle exploding gradient gorulen sequence modellerinde yararlidir.

```
clip_grad_norm_(model.parameters(), max_norm=1.0)
```

43. Hyperparameterlar

Parametre	Etkisi
Learning rate	En kritik training dinamiklerinden
Batch size	Noise, memory, throughput
Depth/width	Capacity
Dropout	Regularization
Weight decay	Parameter shrinkage
Epoch	Optimization suresi
Scheduler	LR zamanlama

44. Debugging sirasi

1. Batch ve target shape kontrol et.
2. Tek batchte overfit etmeyi dene.
3. Loss ve output activation uyumunu kontrol et.
4. Learning rate scan yap.
5. Gradient normlarini izle.
6. Train/validation preprocessing farkini kontrol et.
7. Class distribution ve label mappingi dogrula.
8. Baseline ile karsilastir.

45. Tek batch overfit testi

Kucuk bir batchte modelin lossu neredeyse sifira indirememesi architecture, loss, target, optimizer veya data pipeline bugina isaret edebilir.

46. Common failure modes

- Yanlis target dtype veya shape.
- Sigmoid/softmax ile lossu iki kez uygulamak.
- Eval moduna gecmeden inference yapmak.
- Validation augmentationini training kadar agresif tutmak.
- Learning rate cok yuksek/dusuk.
- Batch normalization istatistiklerini yanlis kullanmak.
- Data leakage.
- Class imbalancei gormezden gelmek.
- GPU kullaniyorum diye data loader bottleneckini gormemek.
- Test setini tuning icin kullanmak.

47. Deep learning ne zaman gereksiz olabilir?

- Kucuk tabular dataset + guclu boosting baseline.
- Is problemi aciklanabilirlik/latency nedeniyle basit model istiyorsa.
- Feature sayisi az ve iliski basitse.
- Compute ve deployment maliyeti kazanci asiyorsa.

Senior bakis
Deep learning kullanmak hedef degil; business problem icin en basit yeterli sistemi kurmak hedeftir.

48. Vaka: tabular churn MLP

Bilesen	Karar
Input	Scaled numeric + categorical embeddings
Architecture	2-4 hidden layer MLP
Loss	Weighted BCE
Metric	PR-AUC + Recall@capacity
Regularization	Dropout + weight decay
Baseline	LightGBM / Logistic Regression
Decision	DL baselinei gecmiyorsa productiona alma

49. Vaka: image classification

Adim	Uygulama
Data	Train/val/test + augmentation
Model	Pretrained CNN/ViT
Training	Transfer learning
Metric	Macro F1 / accuracy
Debug	Class-wise errors
Deploy	Batch/online inference

50. Vaka: text classification

TF-IDF + Logistic Regression baseline kurulur. Sonra embedding veya pretrained Transformer fine-tune edilir. Kazanc accuracy degil, problem metricinde ve latency/cost karsiliginda olculur.

51. Vaka: anomaly detection

Autoencoder normal patterni reconstruction yoluyla ogrenebilir. Reconstruction error yuksek ornekler anomaly adayi olabilir; threshold validation ve business cost ile secilir.

52. Autoencoder sezgisi

Encoder girdiyi latent representationa sikistirir; decoder girdiyi yeniden kurmaya calisir. Bottleneck representation learning ve anomaly detection icin kullanilabilir.

```
x -> encoder -> z -> decoder -> x_hat
loss = reconstruction(x, x_hat)
```

53. Generative deep learninge kopru

- Autoregressive models
- Variational Autoencoders
- GANs
- Diffusion models

Bu modulde hedef
Generative modelling ayrintisina girmek degil; derin ogrenmenin loss + representation + sampling temellerine baglantiyi kurmak.

54. Deep learning ve LLM baglantisi

LLMler de temel olarak differentiable neural networklerdir: embeddings, layers, activations, normalization, residual connections, loss, backpropagation ve optimizerlarla egitilir. Fark buyuk olcek ve Transformer mimarisidir.

55. Production checklist

- Model input schema sabit mi?
- Preprocessing train-serving tutarli mi?
- Checkpoint ve model version kayitli mi?
- Latency ve memory budget olculdu mu?
- Batch size inferencea gore optimize mi?
- Drift/metric monitoring var mi?
- Fallback model/route var mi?
- GPU yokken davranis biliniyor mu?

56. MLOps baglantisi

Deep learning modelleri buyuk artifact, GPU dependency, uzun training ve deney sayisi nedeniyle experiment tracking, model registry ve reproducible environments ihtiyacini daha da belirgin hale getirir.

57. Etik ve risk

- Bias training datasindan buyuyebilir.
- Confidence gercek certainty degildir.
- Hallucination/overconfidence downstream sistemlerde risk yaratabilir.
- Yuksek-stakes kararlarda insan denetimi gerekebilir.

58. Deep learning calisma rutini

9. Basit baseline kur.
10. Data pipelinei dogrula.
11. Kucuk modelle basla.
12. Tek batch overfit testi yap.
13. Learning rate ayarla.
14. Validation curve izle.
15. Regularization ekle.
16. Error analysis yap.
17. Compute-cost kazancini olc.

59. Mülakat sorulari

Soru	Kisa cevap
Neuron nedir?	Agirlikli toplam + bias + activation.
Backpropagation nedir?	Chain rule ile parametre gradientlerini hesaplama mekanizmasi.
Optimizer ne yapar?	Gradientleri kullanarak parametreleri gunceller.
ReLU neden yaygin?	Basit, hizli ve sigmoid/tanha gore gradient akisinda daha avantajli.
Vanishing gradient nedir?	Gradientin derinlik boyunca cok kuculmesi.
Exploding gradient nedir?	Gradient normunun kontrolsuz buyumesi.
Dropout ne yapar?	Trainingde rastgele unitleri kapatip regularization saglar.
BatchNorm vs LayerNorm?	BatchNorm batch statistics; LayerNorm feature boyutunda normalization.
Adam vs SGD?	Adam adaptive ve hizli default; SGD bazi durumlarda guclu generalization saglar.
Weight decay nedir?	Buyuk agirliklari cezalandiran regularization.
Early stopping nedir?	Validation bozuldugunda egitimi durdurmak.
Epoch nedir?	Tum training datasinin yaklasik bir gecisi.
Logit nedir?	Output activation oncesi ham skor.
Transfer learning nedir?	Pretrained modeli yeni probleme adapte etmek.
CNN neden image icin iyi?	Locality ve weight sharing kullanir.
Residual connection ne ise yarar?	Gradient akisina ve derin ag egitimine yardim eder.
Learning rate neden kritik?	Optimization adim buyuklugunu belirler.
Loss vs metric?	Loss optimize edilir; metric performansi raporlar.
Overfit nasil tespit edilir?	Train iyilesirken validation bozulur.
Deep learning her zaman iyi mi?	Hayir; veri, compute, latency ve baselinea gore karar verilir.

60. Uygulama alistirmalari

18. NumPy ile tek neuron forward pass yaz.
19. 2-layer MLP forward passini elle/NumPy ile kur.
20. Sigmoid ve ReLU gradientlerini ciz.
21. Learning rate 1e-1, 1e-3, 1e-5 icin loss curve karsilastir.
22. Dropout 0, 0.2, 0.5 icin validation farkini olc.
23. Weight decay grid deneyi yap.
24. Early stopping patience etkisini incele.
25. Imbalanced classificationda weighted loss dene.
26. Tek batch overfit testi uygula.
27. Pretrained image model ile transfer learning projesi yap.

61. Mini portföy projesi

Proje: Customer Support Intent Classification - Neural Baseline to Production

Cikti	Icerik
01_baseline.ipynb	TF-IDF + Logistic Regression
02_mlp.ipynb	Dense neural baseline
03_training.ipynb	LR, dropout, weight decay experiments
04_error_analysis.ipynb	Class confusion + hard examples
05_transfer.ipynb	Pretrained text encoder/Transformer bridge
src/train.py	Reproducible training script
configs/	Hyperparameter configs
report.md	Metric, latency, cost ve model secim gerekcesi

CV cumlesi

Built and evaluated neural text classifiers with reproducible training loops, regularization, learning-rate scheduling and error analysis, comparing them against a strong classical baseline before selecting the production candidate.

62. Final cheat sheet

Kavram	Hatirlanacak fikir
Neuron	w^T x + b + activation
Forward pass	Input -> prediction
Loss	Optimize edilen hata
Backprop	Gradient hesabi
Optimizer	Parametre update
Learning rate	Adim buyuklugu
ReLU/GELU	Hidden activation
Sigmoid/Softmax	Output probability mapping
Dropout	Stochastic regularization
Weight decay	Parameter regularization
Early stopping	Validationa gore dur
BatchNorm	Batch tabanli normalize
LayerNorm	Feature tabanli normalize
Residual	x + f(x)
Gradient clipping	Exploding gradient kontrolu
Transfer learning	Pretrained bilgiyi aktar
CNN	Spatial/local pattern
RNN/LSTM	Sequential state
Attention	Context-weighted representation
Checkpoint	Model + training state

Siradaki modul

30. baslik PyTorch: tensor, Dataset/DataLoader, model sinifi, loss, optimizer ve tam training loop ile bu teoriyi uygulamaya donusture ceksin.